

## Singly Linked List (Basic Operations)

(Insertion at the end)

```
void insertatend()
```

```
{
```

```
    cout << " ";
```

```
    int val;
```

```
    // value to insert
```

```
    cin >> val;
```

```
    Node* newnode = new Node(val);
```

```
    // creating newnode
```

```
    if (head == NULL)
```

```
    {
```

```
        head = tail = newnode;
```

```
    }
```

```
    // if there is no node before, assign  
    both head & tail → newnode
```

```
    else
```

```
    {
```

```
        tail->next = newnode;
```

```
        tail = newnode;
```

```
    }
```

```
    // if not 1st node, then put it in the next  
    tail & assign tail this newnode.
```

```
}
```

(Insertion at the beginning)

void insertatthebeginning()

```
{
    // Rest of the simple code
    if (head == NULL)
    {
        head = tail = newnode;
    }
    // if first node
    else
    {
        newnode->next = head;
        head = newnode;
    }
    // if not 1st, then move head to the next
    // of this newnode & then point head
    // to this newnode
}
```

(Insertion after value)

void insertaftervalue()

```
{
    if (head == NULL)
    {
        return;
    }
    // if list is empty
```

Node\* temp = head; // To traverse to find  
value without changing head.

```
while(temp != NULL && temp->data != target)
```

```
{  
    temp = temp->next;  
}
```

// moving temp to the target & exiting loop if got NULL or target.

```
if(temp == NULL)
```

```
{  
    return;  
}
```

// to check if loop exits bcz we got NULL so then return (No val found).

```
Node* newnode = new Node(val);
```

// created newnode with new val

```
newnode->next = temp->next;
```

// moving next of temp to the next of newnode as it will be inserted between these two and we have to link them.

```
temp->next = newnode;
```

// now moving newnode in the next of temp.

```
if(temp == tail)
```

```
{  
    tail = newnode;  
}
```

// if temp reaches end, point tail to newnode.

```
{
```

## (Delete by Value)

```
if(head == NULL)
```

```
return;
```

```
}  
// if list is empty
```

```
if(head->data == val)
```

// if value is at head

```
Node* temp = head;
```

// mov head to temp

```
head = head->next;
```

// point head to it's next

```
if(head == NULL)
```

// if head->next was NULL  
(currently head)

```
{
```

```
tail = NULL;
```

// mov ~~tail~~ tail, NULL

```
}
```

```
delete temp;
```

// delet that node

```
return;
```

```
}
```

```
Node* prev = NULL;
```

// to track the <sup>data of</sup> prev node  
of the (to be deleted node)

```
Node* temp = head;
```

// to traverse & find value  
without changing head

// prev will always be <sup>at</sup> 1 node before temp

```
while(temp != NULL && temp->data != val)
```

```
{
```

```
prev = temp;
```

// traversal to find val to  
delete & point temp  
to that node.

```
temp = temp->next;
```

```
}
```

```

if (temp == NULL) // if we get temp == NULL
{
    return it.
}
return;
}

```

prev → next = temp → next;

as prev points to 1 node before temp that is to be deleted so in order to link the prev & next of temp we do this.

```

if (temp == tail) // if temp was tail
{
    tail = prev; // point tail to prev
}

```

```

delete temp; // delete temp
}

```

(Search by Value)

```

bool searchbyvalue()
{

```

```

    if (head == NULL)
    {
        return false;
    }

```

// if no node  
list empty

```

Node* temp = head;

```

// same scene

```

while (temp != NULL)
{

```

// For traversing

```

if (temp->data == val)           // if found data
{
    count--;                     // prompt data
    return true;                 // return true
}
temp = temp->next;               // else, increment temp
return false;                   // if not found return false
}

```

### (Void Update)

```

void update()
{
    ...                          // Prompt & input
    Node* temp = head;          // same scene
    while (temp != NULL)        // traverse
    {
        if (temp->data == val)   // if val to be updated
        {                       found.
            temp->data = newval; // update val
            return;
        }
        temp = temp->next;       // increment
    }
}

```

### (Void Display)

- 1) Simple travers &
- 2) if no node, return

(DRY\_RUN)

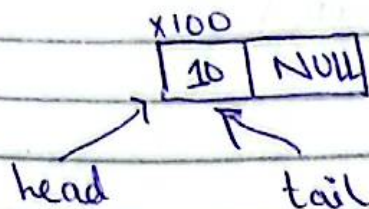
Insertion at end i.e.

⇒ First node entry: (10 enter ⇒ data)

head = NULL

tail = NULL

after new node:



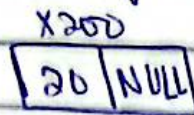
// this is when both head = tail = NULL

⇒ When 2nd or every new entry:

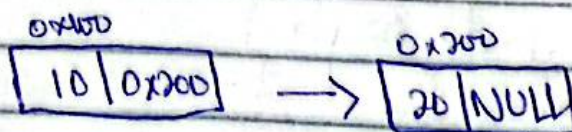
head = 0x100

tail = 0x100

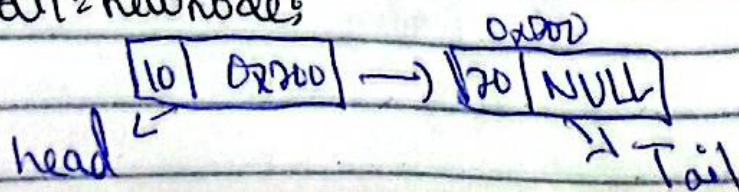
New newnode entry: (20 enter)



tail → next = newnode:



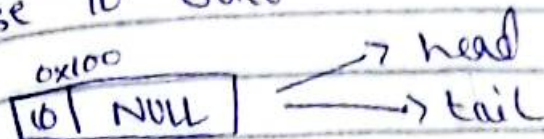
tail = newnode:



insertatbeginning():

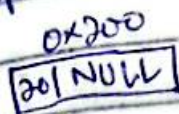
// if (head == NULL) {

When no node before  
suppose 10 enter

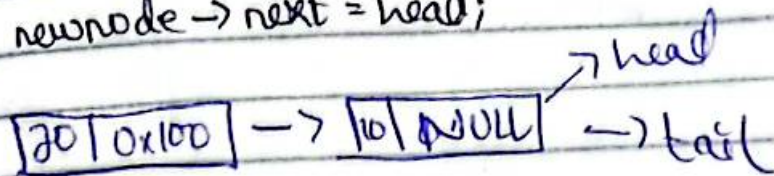


} else {

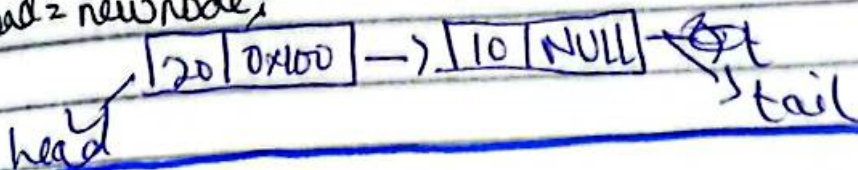
When 2nd or more entries  
suppose 20 enter



Now: newnode → next = head;



head = newnode;



insertaftervalue():

When head = NULL, means no node before  
so return

Node\* temp = head; // so we can traverse through  
the list & find starting from head  
without changing head to find value.

while(1) {

Suppose this is our list

head →

|       |    |       |
|-------|----|-------|
| 0x100 | 10 | 0x200 |
| 0x200 | 20 | 0x300 |
| 0x300 | 30 | 0x400 |
| 0x400 | 40 | 0x500 |
| 0x500 | 50 | NULL  |

tail →

temp → 0x100 // We will traverse it  
we find value (30)

// here it is at data = 10  
temp ≠ NULL.  
// so,

temp = temp → next  
that is 20

// again temp  
at temp = 0x200

temp ≠ NULL ✓  
data ≠ target ✓

loop  
continue

So,

temp = temp → next;

at temp = 0x300

temp ≠ NULL ✓

data = target X

loop exit

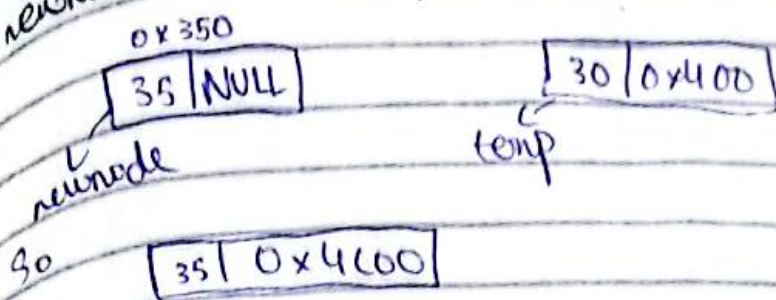
~~if (temp == 0)~~

if (1) // as temp isn't NULL

Note newnode = new Node(Val); say

35 | NULL

newnode  $\rightarrow$  next = temp  $\rightarrow$  next;



temp  $\rightarrow$  next = newnode;

before temp: 30 | 0x400

new: 30 | 0x350

if (temp == tail): not tail

// if tail then also update tail

New linked list is:

|       |    |       |                    |
|-------|----|-------|--------------------|
| 0x400 | 10 | 0x300 | $\rightarrow$ head |
| 0x200 | 20 | 0x300 |                    |
| 0x300 | 30 | 0x350 |                    |
| 0x350 | 35 | 0x400 |                    |
| 0x400 | 40 | 0x500 |                    |
| 0x500 | 50 | NULL  |                    |

tail  $\rightarrow$  0x500

Delete by value():

if (head == NULL): // if no node return

Suppose:

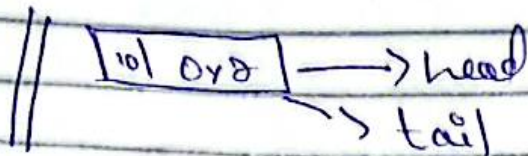
Our list is:

|     |    |      |        |
|-----|----|------|--------|
| 0x1 | 10 | 0x2  | → head |
| 0x2 | 20 | 0x3  |        |
| 0x3 | 30 | 0x4  |        |
| 0x4 | 40 | 0x5  |        |
| 0x5 | 50 | NULL | → tail |

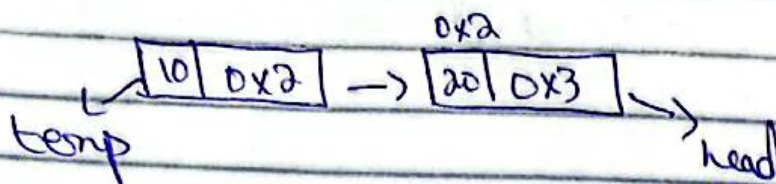
• Lets delete 0x1 → (10)

if (head → data == val) ✓  
{

Node\* temp = head; // new node created



head = head → next



if (head == NULL) X  
{ }

~~delete~~ delete temp;

return;

}

New list is:

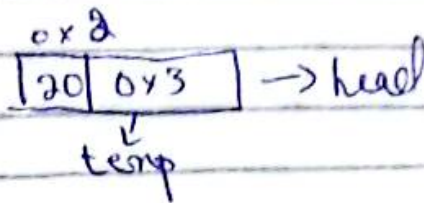
|     |    |      |        |
|-----|----|------|--------|
| 0x2 | 20 | 0x3  | → head |
| 0x3 | 30 | 0x4  |        |
| 0x4 | 40 | 0x5  |        |
| 0x5 | 50 | NULL | → tail |

1) lets delete (30);

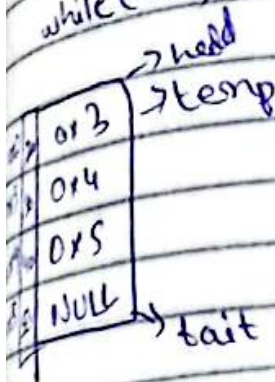
if (head->data == val) X

Node\* prev = NULL;

Node\* temp = head;



while (---)



if (temp == 0x2)

temp = NULL ✓

data = val ✓

temp = temp->next

prev = temp

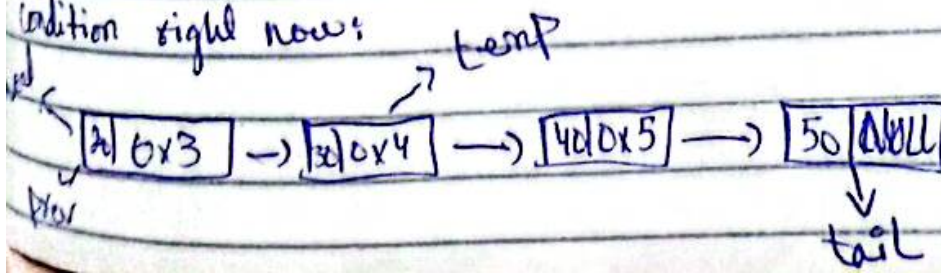
if (temp == 0x3)

temp = NULL ✓

data != val X

exit loop;

condition right now:



if (temp == NULL) {  
3

1) prev -> next = temp -> next;

before: [20|0x3] -> [30|0x4] -> [40|0x5] -> [50|NULL]

Now: [20|0x4] -> [30|0x4] -> [40|0x5] -> [50|N]

4 if (temp == tail) {  
3}

delete temp;

So,

before:

Now: [20|0x4] -> [40|0x5] -> [50|NULL]

2) Void Update();

1) Node\* temp = head;

Suppose; Our linked list is:

|     |    |      |              |
|-----|----|------|--------------|
| 0x1 | 10 | 0x2  | → head, temp |
| 0x2 | 20 | 0x3  |              |
| 0x3 | 30 | 0x4  |              |
| 0x4 | 40 | 0x5  |              |
| 0x5 | 50 | NULL | → tail       |

while(—);

1) def temp = 0x1;

if (temp->data == val) X  
temp = temp->next;

updating  
finding (50) to (25)

1) def temp = 0x2;

if (temp->data == val) X  
temp = temp->next;

1) def temp = 0x3;

if (temp->data == val) ✓  
{  
temp->data = newval;  
return;  
}

So, updated list is:

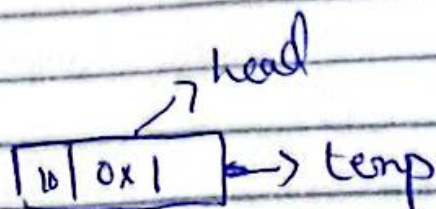
|     |    |      |
|-----|----|------|
| 0x1 | 10 | 0x2  |
| 0x2 | 20 | 0x3  |
| 0x3 | 25 | 0x4  |
| 0x4 | 40 | 0x5  |
| 0x5 | 50 | NULL |

1) bool Searchbyvalue():

lets search 25 in :

if (head == NULL) { } X

Node\* temp = head;



while(1) at temp  $\rightarrow$  0x1

• temp ~~temp~~ ~~temp~~ ✓

(e) if (temp  $\rightarrow$  data = val) X  
temp  $\rightarrow$  next;

at t = 0x2;

if (t  $\rightarrow$  data = val) X  
t = t  $\rightarrow$  ~~data~~ next;

at t = 0x3;

if (t  $\rightarrow$  data = val) ✓  
{

cout << " ";  
return true;

}

}

X = X

void display():

Supposes

|   |      |
|---|------|
| 1 | 0x2  |
| 2 | 0x3  |
| 3 | 0x4  |
| 4 | NULL |

→ head / temp  
~~Node\*~~ temp = head  
→ tail

while (temp  $\neq$  NULL)

{  
cout << temp  $\rightarrow$  data;

temp = temp  $\rightarrow$  next;

}